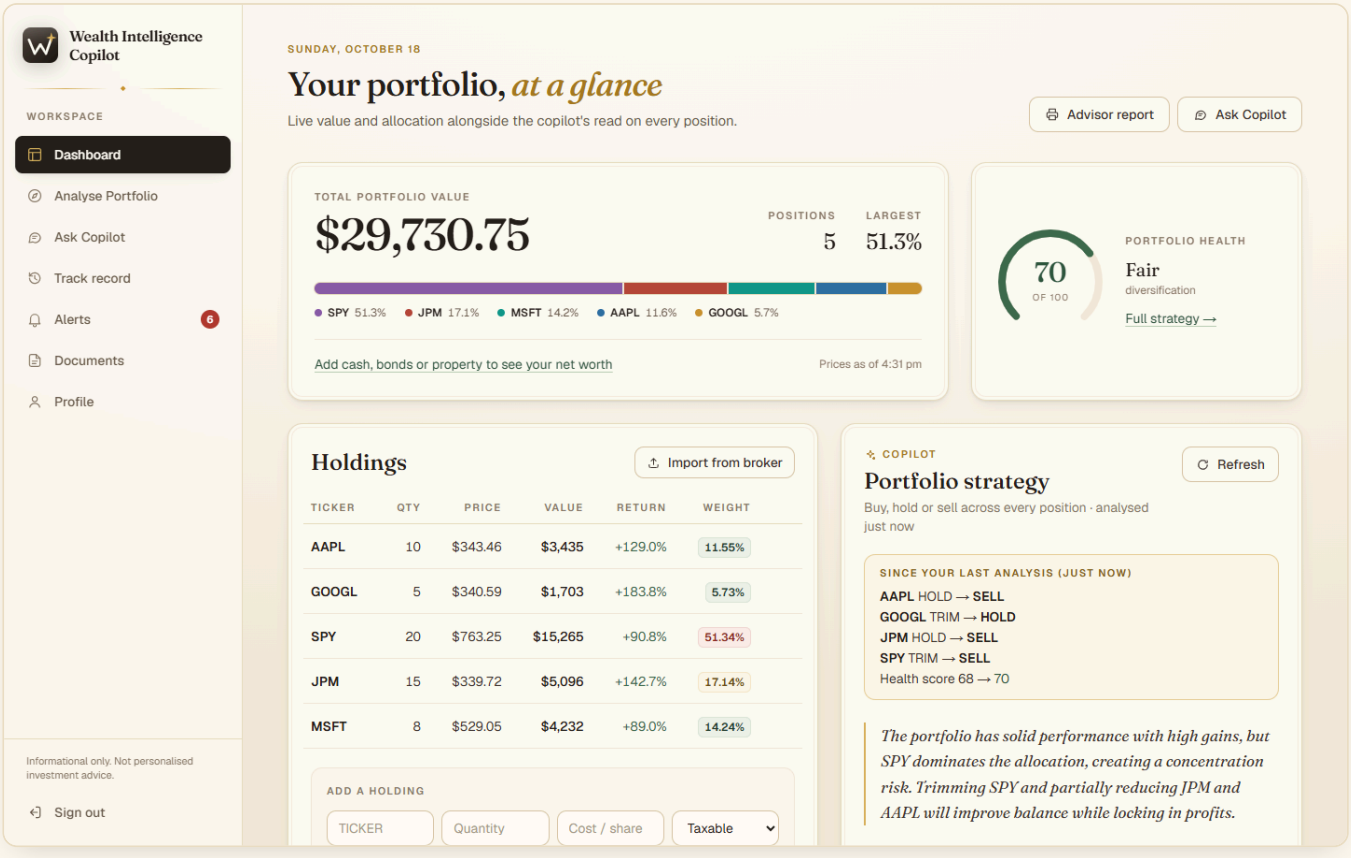
CTO TECHNICAL BRIEF · V4 · OCTOBER 2026

An investment copilot that *shows its work.*

A multi-agent AI system that reads official SEC filings, live market data and a user's real holdings, then explains every Buy, Hold or Sell with citations — personalised to how that person invests, and accountable for how its past calls played out.

7 AI AGENTS · LANGGRAPH | EVIDENCE-GROUNDED RAG | 231 AUTOMATED TESTS · CI | 28 API ENDPOINTS



7	5	0.24 s	~10k
SPECIALISED AGENTS	DATA SOURCES FUSED	PORTFOLIO API, WARM CACHE	LINES OF PRODUCTION CODE

Confidential · For evaluation purposes only

Not investment advice · Not a registered investment adviser

1 · EXECUTIVE SUMMARY

Analyst-grade reasoning for *every* investor

Wealth Intelligence Copilot (WIC) turns a personal brokerage portfolio into a continuously analysed, conversational decision-support system. Users ask in plain English; coordinated AI agents gather primary-source evidence, weigh it against the user's own holdings and risk profile, and return a reasoned answer whose every point can be traced to its source.

The problem

Self-directed investors choose between raw data (filings, charts, news) they don't have time to read, and one-word verdicts they can't verify. Generic AI chatbots don't know their holdings, cite inconsistently, and sound equally confident when right and wrong.

The solution

Specialist agents do the analyst's homework — 10-K financials and risk factors, live price action, news, portfolio concentration — and a composer synthesises it into a direct answer with positives, risks, a confidence level and a citation on every point.

Why now

Fast, low-cost open-weight models (gpt-oss on Groq), free structured SEC data (XBRL), and agent orchestration frameworks (LangGraph) make per-question multi-agent analysis economically viable for individual investors for the first time.

What makes WIC different

Grounded, not generated

Figures shown to the user come from SEC XBRL data and market feeds, never from the language model. Narrative points carry citations such as *MSFT 10-K FY2026, Item 1A Risk Factors*, linked to EDGAR.

Portfolio- and person-aware

Every answer knows the user's weights, their own concentration limit, risk tolerance, horizon, and assets outside the portfolio (cash, bonds, property).

Honest about uncertainty

A deterministic agreement score caps confidence when agents disagree and surfaces "Agents disagree". If the AI fails, the user sees an error — never a fabricated answer.

Accountable over time

Every call is journaled. A track record shows price-then vs price-now for each call; strategy changes are diffed and alerted. Trust is earned, not asserted.

Status at a glance

DIMENSION	STATE TODAY
Product	Working end-to-end application: landing & auth, dashboard, chat copilot, portfolio strategy, profile & net worth, alerts, track record, broker import, document upload, advisor report, phone layout.
AI system	7 agents (5 LLM-based, 2 deterministic) orchestrated with a supervisor router and a LangGraph decision graph; tiered models (gpt-oss-20b / 120b on Groq).
Engineering	~4,550 lines of backend Python, ~5,300 lines of TypeScript, 231 automated tests (124 backend, 107 frontend), GitHub Actions CI on every push, documented modules and a schema-drift test.
Gap to pilot	Execution work, not research: production hosting, brokerage aggregation, answer-stability hardening, hybrid vector retrieval, and regulatory positioning (see Section 13).

2 · PRODUCT TOUR

Features and the *benefit* behind each

A Next.js application with eleven screens. Every feature maps to backend services and, where reasoning is needed, to specific agents.

Ask Copilot — one chat for everything

- **Single stocks**: "Should I buy more Apple?" → direct answer, Buy/Hold/Sell, positives vs risks, key figures, confidence.
- **Comparisons**: up to 3 stocks side by side with one verdict.
- **Whole portfolio** and **retirement** questions routed to dedicated agents.
- Resolves company names and follow-ups ("should I sell *it*?") from context; asks back when unclear.

Benefit: the research an analyst does before a trade, in seconds, shown in full.

Dashboard

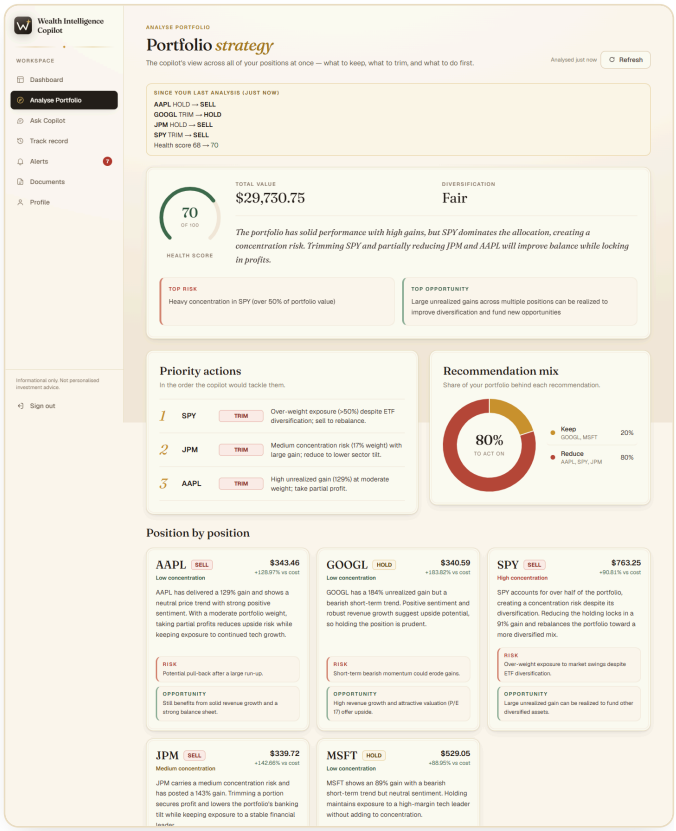
- Live value, net worth, allocation bar, concentration donut labelled against the user's own limit.
- Holdings table with live price, value, return vs cost, weight; inline add/remove.
- AI strategy panel with "since your last analysis" changes; freshness labels ("Prices as of 4:31 pm").

Benefit: a two-minute check-in replaces a spreadsheet.

Analyse Portfolio

- Health score (0–100), diversification, top risk and opportunity.
- Ranked priority actions; recommendation-mix donut showing how much of the portfolio the strategy asks you to act on.
- Position cards with reasoning, key risk, key opportunity and 10-K sources.

Benefit: the whole portfolio's balance, and exactly what to do first.



Analyse Portfolio on the demo account: change summary, health score, priority actions, recommendation mix.

Profile & net worth

Risk tolerance, investment horizon, maximum position size, plus cash, bonds and property. Feeds every agent and every risk label.

Benefit: a cautious 60-year-old and an aggressive 30-year-old get different advice for the same portfolio.

Alerts

Position over the user's limit, a 5%+ daily drop, a new 10-K/10-Q, earnings within 14 days, or a strategy change. Each links to a ready-made question.

Benefit: the copilot tells you when it matters.

Track record

Every call with price then vs now and a verdict: On track, Against the call, Flat, Too early.

Benefit: users can judge the advice before acting on new advice.

Broker import

CSV exports parsed directly (column aliases, title lines skipped); PDF statements extracted by the LLM. Preview-and-confirm; add or replace.

Benefit: onboarding in one upload, not twenty form entries.

Documents

Upload private research (PDF/TXT/CSV, 5 MB). Passages mentioning a company are cited alongside the filings.

Benefit: the copilot reasons over *your* research too.

Advisor report & mobile

One-page print/PDF summary for an advisor or partner; responsive layout with a phone menu and alert bell.

Benefit: shareable, and usable anywhere.

Who it serves, and *how it is different*

Use cases

USER	JOB TO BE DONE	WIC CAPABILITY
Self-directed investor (core)	"Before I buy or sell, is this sensible for *my* portfolio?"	Stock answers with filings, concentration impact, profile fit
Pre-retiree	"Am I on track? How much more must I save?"	Retirement agent using real holdings, other assets and horizon
Financial advisor (B2B2C)	Prepare for client meetings; explain recommendations	Advisor report, cited reasoning, audit journal
Brokerage / wealth platform	Add an AI research layer to existing accounts	API-first backend (28 endpoints), white-label front end
Investor education communities	Teach how to read filings and judge risk	Answers that quote 10-K sections and show positives vs risks

Capability comparison by category

CAPABILITY	WIC	ROBO-ADVISORS	BROKERAGE RESEARCH TOOLS	GENERAL AI CHATBOTS
Conversational Q&A about your own holdings	**Native**	No	Emerging	If pasted in
Answers cite primary sources (SEC filings)	**Every point**	No	Analyst reports	Inconsistent
Knows your weights and your own risk limit	**Yes**	Managed assets only	Own accounts	No
Personalised to risk tolerance and horizon	**Every agent**	**Questionnaire**	Partial	Per chat
Shows reasoning: positives vs risks	**Yes**	Black box	Partial	**Yes**
Flags disagreement and caps confidence	**Enforced in code**	—	No	Rarely
Track record of its own calls	**Yes**	No	No	No
Proactive alerts (filings, earnings, limits)	**Yes**	Account events	**Yes**	No
Works with holdings at any brokerage	**Import CSV/PDF**	Own platform	Own platform	Manual

Category generalisations based on typical products; individual offerings vary.

> **Positioning:** the closest analogy is a research analyst with a Bloomberg terminal who knows your portfolio — delivered at software cost, with every claim traceable. Robo-advisors manage money; chatbots talk; WIC *reasons over evidence about your money and shows its work.*

Business model options

Consumer subscription

Freemium: limited questions free; unlimited analysis, alerts and reports on a monthly plan.

Advisor seats

Per-advisor licence for meeting prep, client reports and audit trail.

Platform / API licensing

White-label copilot for brokerages and wealth platforms over the existing REST API.

The 7-layer *agentic* stack

Each layer has one responsibility and maps to a concrete module in the repository, so layers can be replaced or scaled independently.

LAYER 7	Memory & feedback loop	Decision journal · strategy diffs · alerts engine · track record services/history_service · alert_service · decisions table
LAYER 6	Knowledge & data	SEC EDGAR (XBRL + 10-K text) · Finnhub · Alpha Vantage · user docs · broker files services/sec_service · evidence_service · document_service · import_service
LAYER 5	Retrieval & context assembly	Section extraction · passage ranking · citations · investor profile context sec_service.relevant_passages · profile_service.describe_profile
LAYER 4	Specialist agents	Portfolio · Evidence · Market Data · Portfolio Strategy · Retirement agents/*_agent.py — parallel execution, isolated DB sessions
LAYER 3	Orchestration & synthesis	Supervisor routing · LangGraph decision graph · Composer · guardrails agents/supervisor_agent · graph · recommendation_composer
LAYER 2	API gateway & trust	FastAPI · JWT · Pydantic validation · CORS · security headers · per-user isolation api/* (28 endpoints) · core/security
LAYER 1	Experience	Next.js 16 / React 19 · chat · dashboard · charts · alerts · reports · mobile frontend/app (11 routes, 40 components) · frontend/lib (hooks, API client)

Request lifecycle

A question enters at Layer 1, is authenticated and validated at Layer 2, classified and routed at Layer 3, answered by Layer 4 agents using context assembled at Layer 5 from sources in Layer 6, synthesised and checked back at Layer 3, and recorded at Layer 7 — which in turn feeds alerts, "what changed" and the track record back to Layer 1.

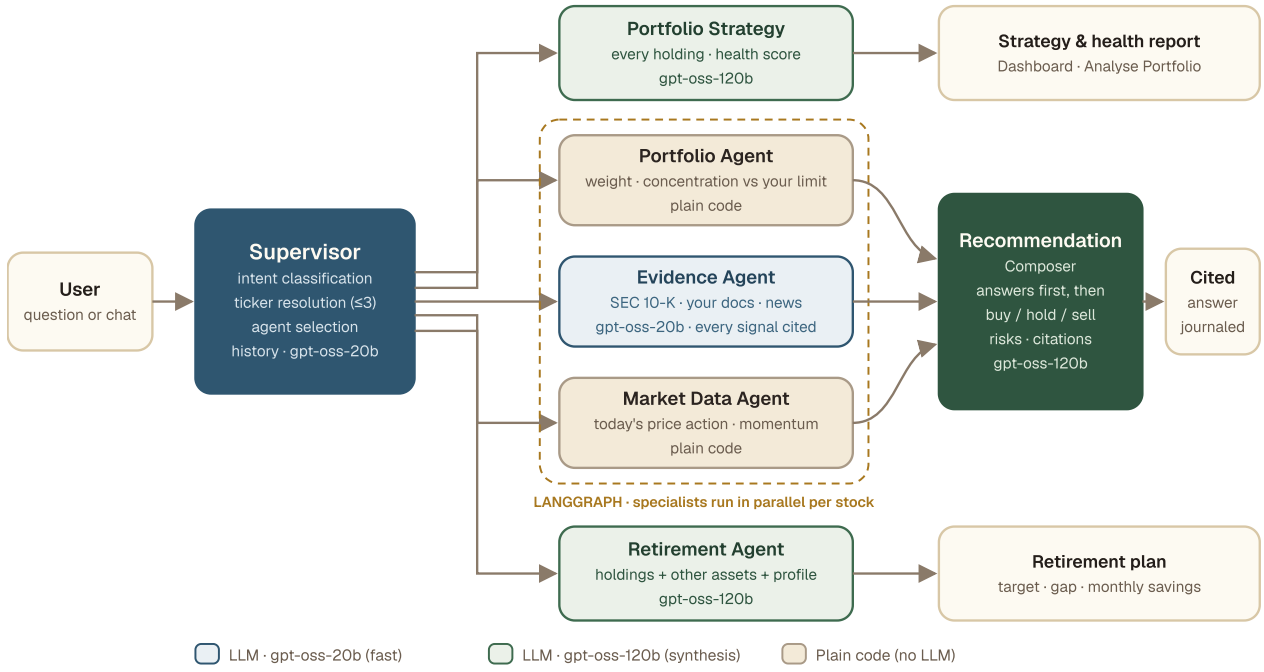
Separation of concerns. HTTP routers contain no business logic; repositories are the only code that queries the database and always scope by user; services own integrations; agents own reasoning.

Deterministic where possible. Two of seven agents and the agreement scorer are plain code: they restate numbers exactly, add zero latency and spend no model budget.

5 · ORCHESTRATION

The 7-agent *agentic AI* architecture

Seven agents, each with one job. A supervisor reads the question and wakes only the agents it needs: stock questions go through three specialists that run in parallel inside a LangGraph graph and are synthesised by a composer; whole-portfolio and retirement questions go to dedicated agents. Two agents are plain code: they handle exact numbers, so they are instant, free and cannot hallucinate.



The seven agents

AGENT	TYPE	RESPONSIBILITY	INPUTS / TOOLS
Supervisor	LLM · 20B	Classifies intent (single position, portfolio-wide, retirement, research, unclear); resolves up to 3 tickers from names, holdings and the last 6 turns; picks which specialists run.	Question, history, held tickers
Portfolio	RULE-BASED	Exact weight and HIGH/MEDIUM/LOW concentration against the user's own limit; impact of buying or selling.	Live valuation, profile
Evidence	LLM · 20B	Reasons over SEC financials, 10-K passages, news sentiment and user documents; returns supporting and contradicting signals, each with a citation.	SEC EDGAR, Finnhub, documents
Market Data	RULE-BASED	Interprets today's price action: signal, momentum, position in the day's range.	Finnhub quote
Recommendation Composer	LLM · 120B	Answers the question directly, then Buy/Hold/Sell with key points, positives, risks and confidence; fits the action to the investor profile.	All specialist outputs, agreement score
Portfolio Strategy	LLM · 120B	A call for every holding, health score, diversification, top risk/opportunity, ranked priority actions.	All positions, evidence per ticker, profile, other assets
Retirement	LLM · 120B	Retirement readiness with real numbers (target nest egg, gap, required savings); designed not to re-ask details already given.	Holdings, other assets, profile, 12-turn history

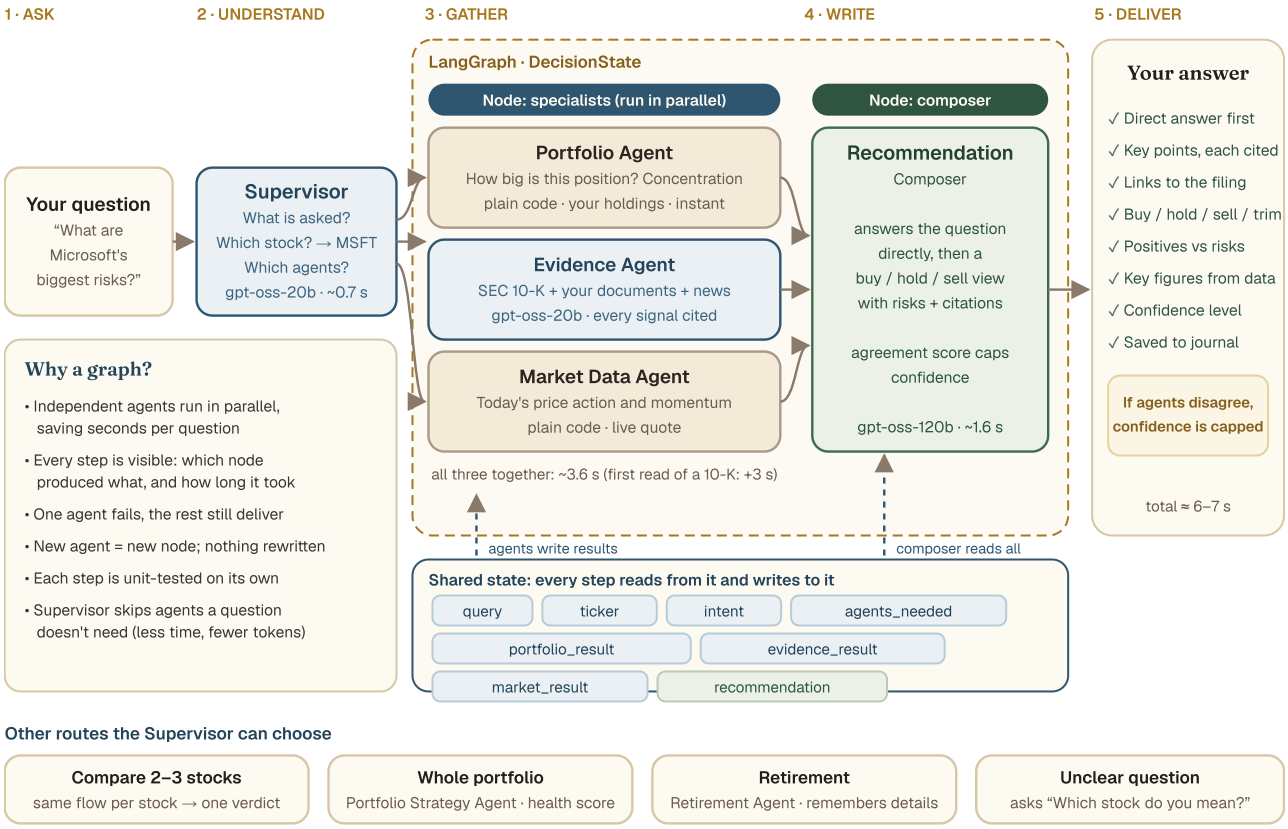
Typed state. `DecisionState` (TypedDict) carries query, ticker, user, intent, `agents_needed`, each specialist's result and the final recommendation. Every hop is observable and unit-tested; a failed specialist returns `{"error"}` into state instead of crashing the graph.

Model tiering. The fast 20b model handles routing, evidence reading and comparison verdicts; the 120b model is reserved for synthesis (composer, strategy, retirement). Both run on Groq's OpenAI-compatible endpoint, so a model swap is one constant.

6 · REQUEST WALKTHROUGH

How a question becomes *an answer*

LangGraph runs the workflow as a graph: steps are nodes, arrows say what runs next, and one shared, typed state carries the data between them. Below, one real question traced end to end, with timings measured on the demo account.



Step by step

STEP	WHAT HAPPENS	WHERE IN THE CODE	TIME
1 · Ask	The question is sent with the last 12 conversation turns, so follow-ups like "should I sell it?" keep their context. Structured answers are converted back to text for the history.	`lib/useChat.ts`	—
2 · Understand	The supervisor classifies intent, resolves up to 3 tickers from company names, holdings and history, and lists the agents needed. Each ticker is verified (held, or has a live price) before anything runs.	`supervisor_agent.py`, `ask_service.py`	~0.7 s
3 · Gather	The specialists run in parallel threads, each with its own database session. The evidence agent retrieves 10-K passages, XBRL figures, news and the user's documents, then reasons over them with every signal cited.	`graph.specialists_node`	~3.6 s (+3 s on a first 10-K read)
4 · Write	The composer answers the question first, then gives the action with positives, risks and citations, fitted to the investor profile. A deterministic agreement score caps confidence when the specialists disagree.	`recommendation_composer.py`	~1.6 s
5 · Deliver	The answer is journaled (feeding the track record and alerts) and rendered as a card: answer, key points with citations, figures, reasoning and sources.	`ask_service.py`, `RecommendationCard`	< 0.1 s

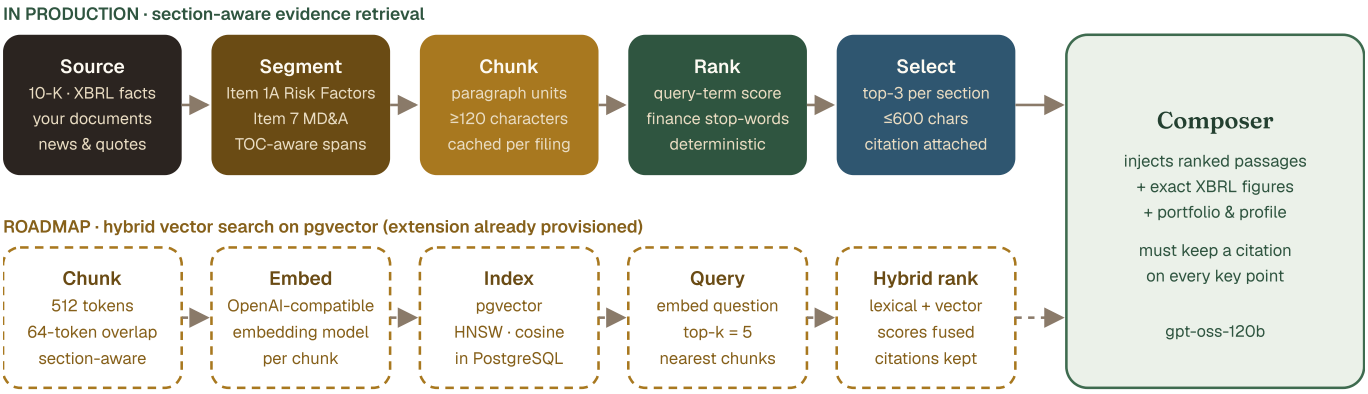
Worked example. "What are Microsoft's biggest risks?" → supervisor: *single position, MSFT, all three specialists* → evidence agent pulls Item 1A Risk Factors passages ranked for "risks" plus three years of XBRL financials → composer leads with the risks, each tagged *MSFT 10-K FY2026, Item 1A Risk Factors*, then a Hold/Trim view weighted by the user's position size and limit.

When something fails. A failed specialist writes `{"error"}` into the state; the composer proceeds with the others and the agreement score ignores the missing signal. If the composer itself fails, the user gets an explicit error with a one-tap **Try again** — never a fabricated answer. Long sessions renew automatically, so a follow-up is never lost to an expired login.

7 · RETRIEVAL-AUGMENTED GENERATION

RAG pipeline: *chunking, retrieval & grounding*

WIC's retrieval is built for financial precision. Numbers come from structured XBRL data; narrative evidence comes from the exact 10-K sections that matter, ranked paragraph by paragraph against the user's question; every retrieved item travels with its citation into the prompt and into the answer.



Numbers bypass retrieval: 12 XBRL metrics × 3 fiscal years (full-year 10-K values only, stale tags dropped) are inserted exactly, and the figures in the UI come from data feeds — never from the model. The roadmap lane adds recall for paraphrase without giving up exact citations.

Chunking strategy

Filings are converted from HTML to text (scripts, styles, `<head>` and hidden inline-XBRL headers removed; block tags become line breaks). Sections are located by their "Item" headings; because the table of contents repeats every heading, the **longest start→end span** is taken as the real section (capped at 150k characters). Sections are then chunked at **paragraph boundaries** — the natural semantic unit of filing prose — discarding fragments under 120 characters (headers, page numbers, table debris).

Retrieval strategy

The question is reduced to content terms (3+ letters) after removing a finance-aware stop-word list that strips intent words ("buy", "sell", "hold", "position", "stock") which would otherwise match every paragraph. Paragraphs are scored by term frequency, ties broken by document order, and the top three per section are kept (each ≤600 characters, cut at a word boundary). Generic questions fall back to the section's opening paragraphs.

Sources fused per answer

SOURCE	RETRIEVAL
SEC XBRL facts	Revenue, gross/operating/net income, EPS, cash flow, capex, cash, assets, liabilities, debt, equity — 3 fiscal years
10-K text	Risk Factors and MD&A passages ranked to the question
User documents	±150-character windows around ticker/company mentions; one per document, up to 4 — favours source diversity
Market & news	Quote, trend, fundamentals, 7-day headline sentiment; DB-cached 10 min
Portfolio & profile	Live weights, user limits, risk tolerance, horizon, other assets

Why lexical-first for filings. (1) **Precision**: the section is located exactly, so a question about risks reads Item 1A, not a semantically similar paragraph from elsewhere. (2) **Freshness**: the latest filing is fetched on demand — there is no index to go stale. (3) **Auditability**: retrieval is deterministic and reproducible, which matters for explainability in financial advice. (4) **Cost and latency**: no embedding calls, no vector index to maintain.

Next step — hybrid retrieval. PostgreSQL already runs with the `pgvector` extension provisioned. The roadmap adds section-aware embeddings for user documents and long filings, combined with the current term scoring (hybrid lexical + vector ranking) to catch synonyms and paraphrase ("liquidity" vs "cash position") while keeping exact citations.

8 · TRUST BY DESIGN

Guardrails, enforced *in code*

Prompt instructions are suggestions to a model; WIC's critical guarantees are implemented as code paths with tests.

GUARDRAIL	MECHANISM	ENFORCED BY
Agent-agreement score	Deterministic score (0–1) from the portfolio, evidence and market signals; skipped or failed agents are excluded rather than counted as disagreement. Below 0.4, confidence is forced low and "Agents disagree" is shown.	Code + 6 tests
Data, not model, for numbers	Prices, values, weights, returns and SEC figures shown in the UI come from feeds; concentration flags are recomputed from live weights and the user's limit, overriding whatever the model wrote.	Code + tests
Citations on every point	Retrieved passages carry labels such as *AAPL 10-K FY2025, Item 1A Risk Factors*; the composer must keep a citation per key point; sources link to EDGAR.	Prompt + UI
No fake answers	Any agent failure returns an explicit error (HTTP 502) — a placeholder recommendation is never shown or journaled.	Code + tests
Ticker verification	A ticker runs only if the user holds it or it has a live market price; otherwise the copilot asks which company was meant.	Code + tests
Profile fit	The composer and strategy agents receive the investor profile and must not suggest positions above the user's stated maximum.	Prompt + tests
Human-in-the-loop data entry	Holdings extracted by the AI from statements are only saved after the user reviews and confirms each row.	Code + tests
Resilience	Empty or truncated model output is retried with double the token budget; short rate limits are waited out; alert checks continue if a provider fails.	Code + tests
Accountability	Every recommendation and strategy is journaled with confidence and agreement score; the track record and "what changed" are built from this audit trail.	Code + tests

Personalisation and engagement architecture

Profile → every agent

A single `get_profile()` supplies risk tolerance, horizon and the user's limit (Medium = 60% of it) to the portfolio agent, composer, strategy and retirement agents, the summary API and the alert engine — so every screen labels risk identically.

Alert engine

Checks concentration, 5%+ daily drops, filings within 14 days and earnings within 14 days, in parallel threads with isolated DB sessions. A unique `(user, dedupe_key)` constraint makes repeated checks idempotent; checks are throttled to once per 30 minutes per user.

Strategy diffing

Each new strategy is compared with the previous journaled one: changed calls, added/removed tickers and health-score movement are attached to the response and raised as an alert.

Track record

Calls are condensed to each ticker's last call per day, then to changes only; prices now are fetched in parallel; verdicts are directional (buy = rise, sell/trim = fall), with "Too early" under 3 days.

An *API-first* backend: 28 endpoints

The web app is one client of a documented REST API (FastAPI, OpenAPI schema generated automatically). Partners can embed the same capabilities.

AREA	ENDPOINT	PURPOSE
Auth	`POST /auth/signup` · `/auth/login` · `GET /auth/me`	Account creation, JWT issue, current user
Copilot	`POST /decisions/ask`	Natural-language question → `route`: recommendation, comparison, strategy, retirement or clarify
	`POST /decisions/query` · `/decisions/retirement`	Explicit-ticker analysis; retirement analysis with history
	`GET /decisions/portfolio-strategy`	Strategy for every position, with changes since the last one
Accountability	`GET /decisions/journal` · `/decisions/track-record`	Audit trail; past calls with price then/now and verdict
Portfolio	`GET/POST /portfolio/holdings` · `DELETE /portfolio/holdings/{id}`	Holdings management
	`GET /portfolio/summary`	Live valuation, net worth, user limits, price freshness; correlation on request
	`POST /portfolio/import/preview` · `/portfolio/import`	Broker CSV/PDF extraction without saving; confirm (add or replace)
Profile	`GET/PUT /profile` · `GET/POST /assets` · `DELETE /assets/{id}`	Investor profile; other assets and total
Alerts	`GET /alerts` · `POST /alerts/read` · `/alerts/{id}/read`	Alerts with unread count (runs checks when due); read state
Evidence & docs	`GET /evidence/{ticker}` · `POST /documents/upload` · `GET /documents/` · `DELETE /documents/{id}`	Raw evidence; private document library
Ops	`GET /health`	Liveness for load balancers

Example: one call, one structured answer

Request

```
POST /decisions/ask
{ "query": "Is my MSFT position too large?",
  "conversation_history": [ … ] }
```

Response (abridged)

```
{ "route": "recommendation", "ticker": "MSFT",
  "action": "hold", "confidence": 0.7,
  "answer": "…", "key_points": [{ "point": "…",
    "citation": "MSFT 10-K FY2026, Item 1A" }],
  "evidence_for": […], "evidence_against": […],
  "metrics": { "price": …, "weight": … },
  "disagreement_flag": false, "sources": […] }
```

Full *tech stack*

FRONTEND	BACKEND	ORCHESTRATION	INFERENCE	DATABASE
Next.js 16	**FastAPI**	**LangGraph**	**Groq API**	**PostgreSQL**
TypeScript · React 19 · Tailwind 4	Python 3.11 · Uvicorn · Pydantic v2	LangChain core · TypedDict state	gpt-oss-20b · gpt-oss-120b	pgvector provisioned · SQLAlchemy 2

AUTH	MARKET DATA	PARALLELISM	DEPLOYMENT	QUALITY
JWT HS256	**Finnhub**	**ThreadPoolExecutor**	**Railway + Vercel ready**	**pytest + Vitest**
bcrypt · sliding sessions	SEC EDGAR · Alpha Vantage (historical)	max_workers=10 · 5-min price cache	Docker DB · GitHub Actions CI	231 tests · CI on every push

External API integrations

API	DATA USED	CACHING	NOTES
Finnhub	Real-time quotes, daily change, company profile, P/E and growth metrics, 7-day news, earnings calendar	Prices 5 min (memory) · evidence 10 min (DB) · earnings 12 h	All holdings priced in parallel (`ThreadPoolExecutor`, `max_workers=10`); simultaneous requests for one ticker share a single fetch; last known price used if a fetch fails.
SEC EDGAR	Ticker→CIK map, XBRL company facts, latest 10-K/10-Q, filing documents	Facts 24 h (DB) · 10-K sections per filing (memory)	Free, no key; requires a contact User-Agent. Ticker map pre-loaded at startup; holdings' 10-Ks pre-loaded in the background after each strategy run.
Alpha Vantage	Historical daily prices for the correlation matrix	12 h	Only on request (`?include_correlation=true`) — removed from the dashboard's critical path, cutting load time to sub-second.
Groq API	LLM inference: gpt-oss-20b (routing, evidence, comparisons) and gpt-oss-120b (composer, strategy, retirement, statement import)	Strategies reused for 30 min when nothing changed	OpenAI-compatible endpoint — a model swap is one constant. Empty or truncated replies retried with a larger budget; short rate limits waited out.
PostgreSQL + pgvector	Users, holdings, profiles, other assets, decisions (audit trail), documents, evidence cache, alerts — 8 tables	—	One database for relational data and (next) vectors; no separate vector store to operate. Schema checked against the ORM models by a test.

Measured performance (demo account, local)

0.24 s	~6–7 s	instant	10–20 s
PORTFOLIO API, WARM	TYPICAL COPILOT ANSWER	DASHBOARD STRATEGY ON REVISIT (REUSED ≤30 MIN)	FRESH FULL-PORTFOLIO STRATEGY

Model calls are fast (supervisor ~0.7 s, evidence reasoning ~1.2 s, composer ~1.6 s); most remaining time is data retrieval, which caching and background pre-loading of filings reduce. Reusing an unchanged strategy also stops the dashboard competing with chat for AI capacity — under contention a question slowed from ~6 s to ~15 s.

Security posture — *implemented and planned*

CONTROL	IMPLEMENTATION	STATUS
Authentication	JWT (HS256) on every protected route, validated server-side per request; expired sessions return users to sign-in. Passwords hashed with bcrypt (8–72 character length enforced at the API).	DONE
Sliding sessions	Tokens are renewed automatically once past half their lifetime (header exposed via CORS), so active users are never signed out mid-task; idle sessions expire after 60 minutes and the login page explains why.	DONE
Data isolation	Every repository query is scoped to the authenticated user (holdings, profile, assets, documents, alerts, journal); cross-user access tests for holdings, documents, assets, alerts and the journal.	DONE
Input validation	Pydantic schemas on every endpoint: lengths, ranges, enumerations (risk levels, asset classes), chat-history roles, upload size (5 MB) and type.	DONE
Transport & headers	X-Frame-Options DENY, X-Content-Type-Options, Referrer-Policy, Permissions-Policy; HSTS over HTTPS.	DONE
CORS	Explicit origin allow-list from configuration.	DONE
Secrets	All keys in environment variables, excluded from git; startup self-check warns on placeholder secrets, short JWT secrets and missing SEC contact.	DONE
Injection	ORM-parameterised SQL; SQL-injection test cases; chat history restricted to user/assistant roles.	DONE
Error recovery	AI failures return an explicit error with a one-tap retry, never a placeholder; a failed specialist doesn't stop the others; duplicate evidence writes are caught (IntegrityError guard returns the existing record); failed price fetches fall back to the last known price or are excluded from totals.	DONE
Test isolation	Tests refuse to run against the application database.	DONE
Session storage	Token currently in browser localStorage → move to httpOnly, SameSite cookies.	PLANNED
Rate limiting	No API-level rate limiting yet → per-user and per-IP limits at gateway.	PLANNED
Content Security Policy	Not yet set → strict CSP for the front end.	PLANNED
Production identity & audit	AWS target: Cognito, RDS encryption at rest, CloudWatch audit logging.	PLANNED

Engineering quality

231
AUTOMATED TESTS

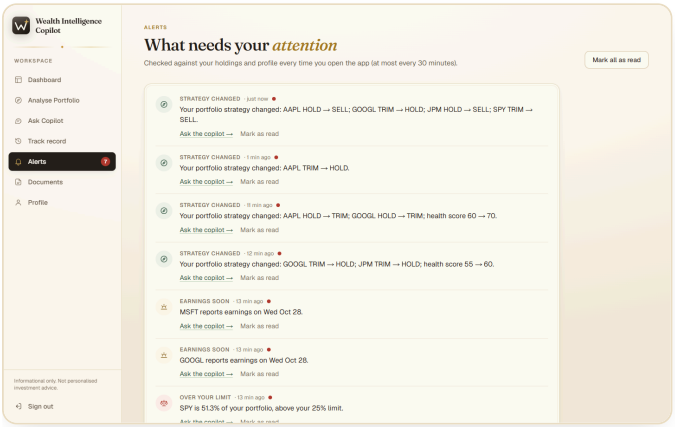
CI
EVERY PUSH & PR

100%
BACKEND FUNCTIONS DOCUMENTED

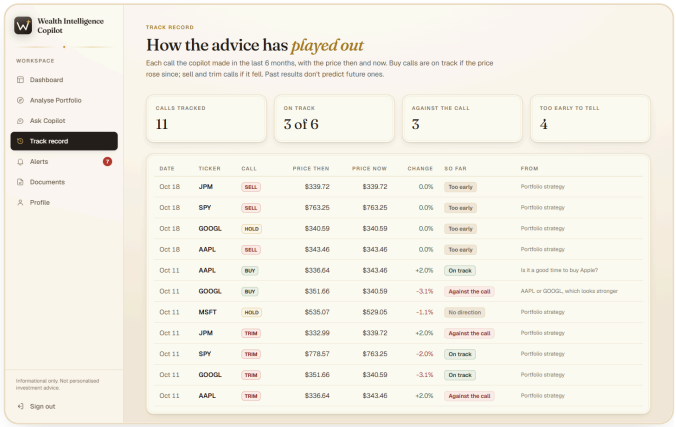
0
API KEYS NEEDED TO RUN TESTS (AI & MARKET DATA MOCKED)

Test suites cover authentication, isolation, security, guardrails, routing, SEC parsing, performance, schema drift, profile, alerts, history, import and personalisation (backend), plus components, charts, formatting and page flows (frontend). CI reproduces a clean install — it caught and fixed a real dependency incompatibility (bcrypt 5 vs passlib) that would have broken production sign-in.

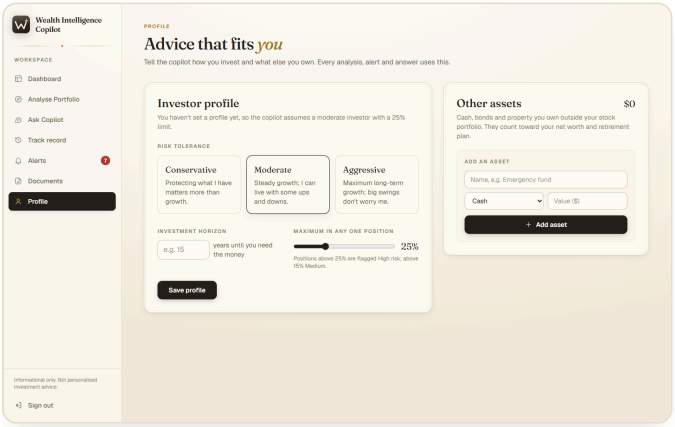
From question to *accountable* decision



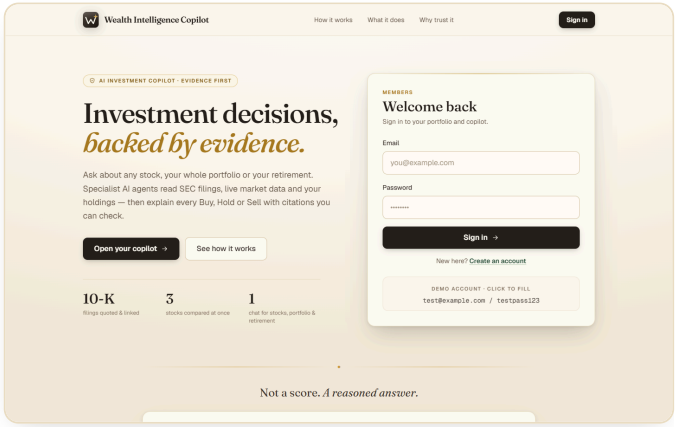
Alerts: limit breach, upcoming earnings, strategy changes — each with a ready-made question.



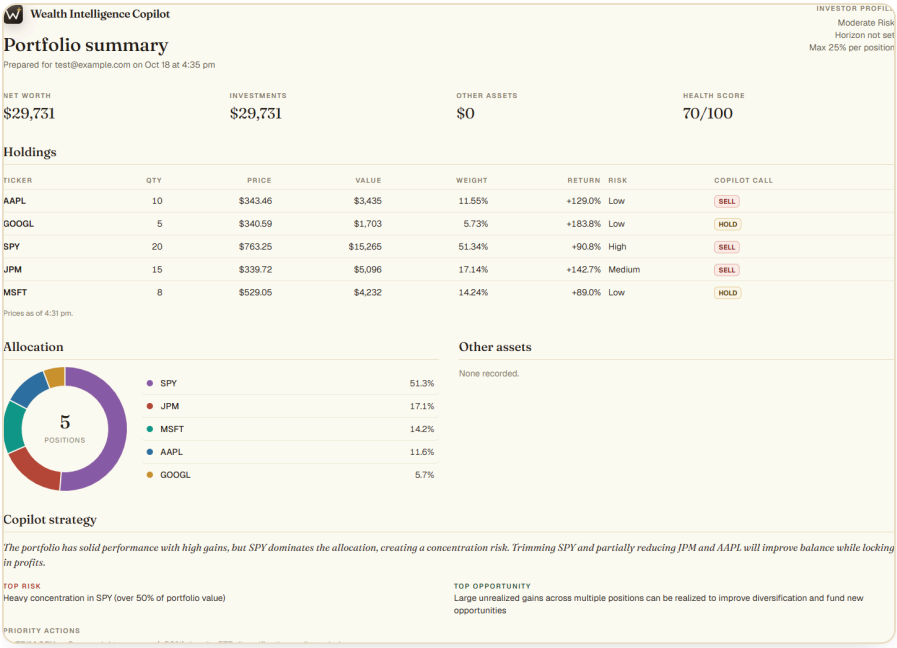
Track record: every call with price then vs now and a verdict.



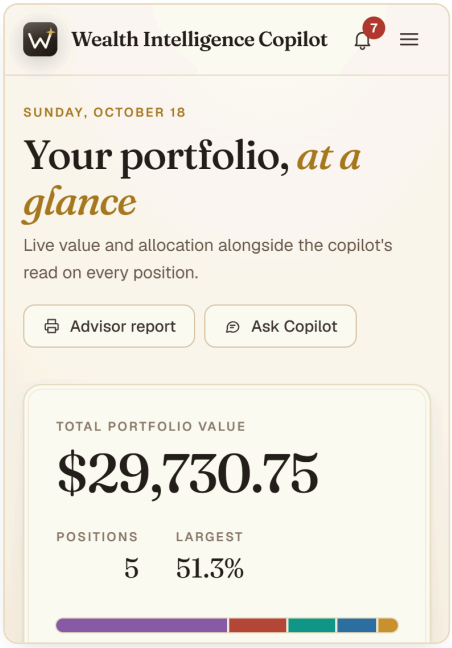
Profile: risk tolerance, horizon, position limit and other assets.



Landing page with sign-in and product explanation.



Advisor report, print view.



Phone layout.

A working system with a *well-scoped* path to pilot

The hard problems — multi-agent orchestration, evidence-grounded retrieval from primary filings, personalisation, accountability and engineering discipline — are solved and tested. What remains is execution.

AREA	TODAY → NEXT	INDICATIVE EFFORT
Answer stability	Partly addressed: an unchanged portfolio now reuses its strategy for 30 minutes, so calls don't shift between visits. Next: deterministic sampling settings, a daily strategy of record, and calls that change only on new evidence.	1–2 weeks
Hybrid retrieval	Lexical section-aware retrieval → add pgvector embeddings for documents and long filings with hybrid ranking.	2–3 weeks
ETF look-through	Index funds are treated as single positions (prompt-level mitigation) → decompose ETF holdings for true exposure.	1–2 weeks
Brokerage aggregation	CSV/PDF import → live account linking via an aggregator (e.g. Plaid / SnapTrade).	4–6 weeks
Notifications	In-app alerts on visit → scheduled checks with email and push.	1–2 weeks
Security hardening	httpOnly cookies, rate limiting, CSP (Section 11).	1–2 weeks
Production deployment	Local / Railway-ready → AWS (ECS Fargate, RDS, S3, Cognito, CloudWatch) with shared cache.	3–4 weeks
Regulatory	Decision support, not advice → RIA registration or white-label under an existing adviser.	Legal process

Effort estimates are indicative, for a small team, and exclude legal timelines.

Strengths for investors

- **Defensible trust layer**: citations, code-enforced guardrails, audit trail and a public track record — hard to retrofit into chat-first products.
- **Low marginal cost**: free primary data (SEC), deterministic agents where possible, small model for routing, large model only for synthesis.
- **Modular agents**: new agents (tax, ETF look-through, ESG) plug into the graph without touching existing ones.
- **API-first**: the same backend serves consumers, advisors and platform partners.
- **Engineering discipline**: 231 tests, CI, documented modules, schema-drift protection.

Risks to evaluate

- **Model consistency** must be hardened before advice is presented as stable (plan above).
- **Regulatory pathway** must be settled before monetising advice-like features.
- **Data licensing** for commercial scale (market data tiers) affects unit costs.
- **Coverage**: SEC data covers US registrants; ETFs and foreign issuers have no 10-K.

> *"Robo-advisors manage money. Chatbots talk. WIC reasons over evidence about your money — and shows its work."*

Source code: github.com/rahulwarekar20/wealth-intelligence-copilot · Figures in this brief are taken from the codebase and from measurements on the demo account in October 2026. Wealth Intelligence Copilot is a decision-support tool; it is not investment advice and not a registered investment adviser.